

AcquisitionOS

AI Market Intelligence and Client Acquisition Platform

A single system that finds businesses, scores them with predictive models, runs contextual outreach across four channels and carries every deal through a measurable pipeline to closure. This dossier documents the mission, architecture, technology stack and delivery discipline behind the product.

Product Engineering Division

Current status: In active development

Table of Contents

1. Mission	1
2. The Problem · Before	1
3. What We Built	2
4. System Architecture	2
5. Data Flow	4
6. Before → After	4
7. Technology Stack	5
8. Delivery Discipline	7
9. Field Notes · Verified	7

1. Mission

AcquisitionOS is an AI market intelligence platform that automates the full client acquisition lifecycle: finding businesses, qualifying them, contacting them and closing the deal. It is built for agencies, consultancies and sales teams that need to run acquisition at scale without growing headcount at the same rate. The product consolidates what used to be a patchwork of maps, directories, spreadsheets and messaging apps into one command center.

Our mission is simple to state and hard to do: make client acquisition a measurable, repeatable engineering process instead of a personality driven art. Every business that can be discovered should be discoverable inside the platform. Every lead should arrive with context and a score a salesperson can act on the same day. Every conversation should be logged, every follow up scheduled and every rupee of pipeline visible to the whole team, not just to the person who opened the account.

The platform is in active development. The core loop of discovery, scoring, outreach and pipeline management is operational in the current build, and the remaining work is hardening, the move to PostgreSQL and launch polish. We state this openly in every client conversation. This document should be read as an engineering dossier on a working system under construction, not as marketing for a finished box.

IN DEVELOPMENT Current project status	1,032 TypeScript source files	326,000+ Lines of code in the repo	53+ Database tables designed
---	---	--	--

2. The Problem · Before

Client acquisition, as most service businesses run it today, is manual at every step. The problems below describe the operating state we built this platform to escape, and we have lived inside each of them ourselves.

Finding businesses to pitch was slow, manual work. Someone scanned Google Maps and trade directories city by city, copied names into a spreadsheet, opened each website in a tab and formed an opinion by hand. Covering twenty two niches across six countries meant the work never finished; it only paused. The moment research stopped, the pipeline stopped with it.

Deciding who deserved attention was guesswork. Nothing in a spreadsheet row says whether a business is about to spend money. Urgency lived in the head of whoever did the research, and it left the company when that person did. Two salespeople looking at the same lead would disagree completely about its value, and there was no shared evidence to settle the argument.

Outreach was generic because personalization takes time nobody had. First messages went out in bulk with a name swapped in. Replies landed in personal inboxes that no manager could see. Follow ups lived in memory and sticky notes, and a deal that went quiet simply stayed quiet. Nothing reminded anyone to return to a warm conversation that had cooled.

Management ran on opinions. There was no funnel to inspect, no conversion rate between stages, no record of who contacted whom and when. Weekly reviews reconstructed the past from recollection. The result was a familiar pattern: high effort, low visibility and quality that depended on who was working rather than on what the process enforced.

3. What We Built

AcquisitionOS is one platform with five working surfaces. Each surface maps to a module in the repository with its own routes, data model and tests. The table below is the honest map of the system, including the state of each part in the current build.

Surface	What it does	State
Lead Intelligence	OSINT discovery across Google Maps, LinkedIn, Yelp and IndiaMART. Automated digital audit of every prospect website. Four composite scores with AI written explanations.	Operational in build
Engagement	Sequence based outreach across Email, WhatsApp, LinkedIn and Instagram. Five template categories with variable interpolation and AI message generation.	Operational in build
Deal Room	Kanban pipeline with drag and drop stages, aging alerts and velocity metrics. AI proposal generation, meetings and calendar integration.	Operational in build
Revenue	Subscriptions, credits ledger, entitlements and payment orders running on Stripe and Razorpay with invoices and coupons.	In hardening
Control	Teams and roles, audit trail, GDPR endpoints, workflow definitions, realtime notifications and a command palette for navigation.	In hardening

Table 1: The five surfaces of the platform and their build state

The scale of what exists is concrete. Discovery runs across 22 business niches and 6 countries. Scoring produces 4 composite scores per lead: Reply Score, Deal Conversion Score, Urgency Score and Revenue Potential Score. Outreach reaches leads on 4 channels from 5 template categories. Underneath, the API surface spans more than 60 route groups and the schema carries more than 50 tables with their relations, indexes and migration annotations already in place.

4. System Architecture

The system is deliberately monolithic in deployment and modular in code. One Next.js 16 application serves the React 19 front end and the REST API through more than sixty route groups, so a single deploy carries the whole product. This choice keeps early velocity high and defers the operational cost of splitting services until scale actually demands it.

Data access goes through Prisma ORM sitting over PostgreSQL in production and SQLite in local development. Redis handles queues, caching and rate limiting. A WebSocket service pushes realtime notifications to the dashboard. AI features, scoring explanations, message generation, proposals and the sales assistant, call GLM

series models through a single SDK facade, so a model change never touches business logic. Python data services run the crawler and enrichment workers that feed the intelligence layer.

Security is layered rather than bolted on. Authentication supports password login, one time passwords, magic links and Google OAuth, all behind signed JWT sessions. Public endpoints are rate limited at the route level, sensitive actions append to an immutable audit trail, and role and organization checks run inside the route handlers before any data leaves the server.

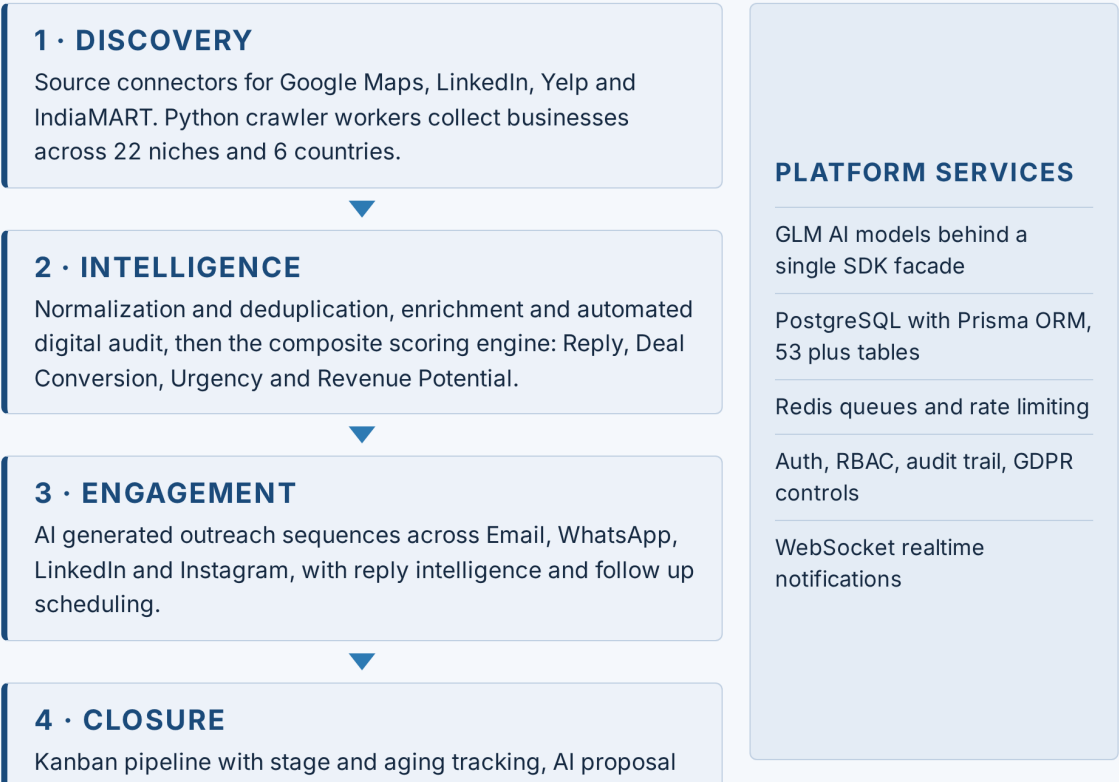


Figure 1: Four stage acquisition flow with platform services on the right

Layer	Owns
Discovery	Source connectors and Python crawler workers. Raw business records with location, category and contact signals.
Intelligence	Normalization, deduplication, digital audit and the composite scoring engine with AI explanations.
Engagement	Sequences, channel adapters, reply capture and classification, follow up scheduling.
Closure	Pipeline stages, proposals, meetings, analytics and reporting.
Platform services	AI models, database and ORM, Redis, authentication and RBAC, audit trail, realtime transport, billing rails.

Table 2: Architectural layers and their responsibilities

5. Data Flow

A lead moves through the system in six defined steps. Each step writes its output to the database and its event to the activity timeline, so the full history of any lead can be reconstructed from records alone, without asking anyone what happened.

Step	Stage	What happens
1	Capture	Source connectors pull businesses with location, category, rating and contact signals from maps and directory sources across the target niches.
2	Normalize	Records are cleaned, deduplicated and keyed so the same business is never worked twice by different people.
3	Enrich	The digital audit fetches each prospect website and grades interface quality, SEO fundamentals and technology footprint. Firmographic fields are completed from source data.
4	Score	The scoring engine computes four composite scores and produces an AI written explanation for each, so every number carries its reasoning.
5	Engage	Scored leads enter sequences. Messages are generated per channel from audit context, scheduled and sent. Replies are captured, classified and answered with suggested follow ups.
6	Close	Engaged leads advance through pipeline stages. Proposals are generated, meetings are booked through the calendar, and analytics aggregate every event into funnels and trend views.

Table 3: The six step lead lifecycle

Two properties matter more than any single step. First, the flow is idempotent at every stage: running a step again does not create duplicate work or duplicate messages. Second, every stage emits events rather than mutating silent state, which is what makes the dashboards, heatmaps and timelines in the analytics module a faithful record instead of a decoration.

6. Before → After

The comparison below is qualitative by design. We will publish measured numbers after the pilot cohort completes; until then we describe direction rather than invent magnitudes. Each row maps to a module listed in section three, at the build state shown there.

Dimension	Before	With AcquisitionOS
Prospecting	Manual scanning of maps and directories, niche by niche, spreadsheet by spreadsheet.	Continuous discovery across 22 niches and 6 countries from inside the platform.
Qualification	Gut feel applied to a spreadsheet row.	Four composite scores per lead with written AI reasoning.
First contact	Generic templates sent in bulk with a name swapped in.	Channel aware messages generated from each prospect audit context.

Dimension	Before	With AcquisitionOS
Follow ups	Memory, sticky notes and personal inboxes.	Scheduled reminders with overdue alerts visible to the whole team.
Deal tracking	Reconstructing history from email threads.	Kanban pipeline with stage aging, velocity metrics and win and loss analysis.
Team visibility	Weekly verbal updates from each salesperson.	Live funnels, score heatmaps and activity timelines.

Table 4: Operating state before the platform and after

None of these rows claim magic. Each one names a specific capability that exists as running code today, and the build state of that capability is stated in Table 1. That traceability, from promise to module to state, is the honest way to present a product that is still in development, and it is how we prefer to be judged.

7. Technology Stack

The stack was chosen for hiring reality and long term maintainability, not for novelty. Every item below is in the repository today, with the version pinned in the manifest and the integration exercised by the running build.

Frontend	Role
Next.js 16 with App Router	Application framework and routing
React 19	UI runtime
TypeScript 5	End to end typing across UI and API
Tailwind CSS 4	Styling system with design tokens
shadcn and Radix UI	Accessible component primitives
Recharts	Dashboards, funnels and heatmaps
Framer Motion	Interaction and transition layer
zustand	Lightweight client state
react-hook-form with Zod	Forms and schema validation at the edge

Table 5: Frontend stack

Backend and data	Role
Next.js API routes	REST service surface, 60 plus route groups
Prisma ORM 6	Database access, migrations, typed client
PostgreSQL (Supabase)	Production database, 53 plus tables
SQLite	Local development database with identical schema
Redis via ioredis	Queues, caching and rate limiting
NextAuth and JWT (jose)	Sessions, refresh and signed tokens

Backend and data	Role
bcryptjs	Password hashing
Nodemailer and Resend	Transactional email delivery
WebSocket service	Realtime notifications and presence

Table 6: Backend and data stack

Python powers the data services layer. The discovery and enrichment workers are separate Python processes so heavy crawling, parsing and batch transforms never compete with the request path of the main application. They expose small internal APIs, are scheduled independently and write their results back through the same database contracts the platform uses.

Python data services	Role
Python 3.11	Service runtime for crawler and enrichment workers
FastAPI with Uvicorn	Internal service API for worker control
httpx	Concurrent HTTP client for source fetching
BeautifulSoup4 with lxml	HTML parsing for website audits
pandas	Batch enrichment and tabular transforms
Pydantic	Configuration and payload validation
APScheduler	Crawler runs and recurring job scheduling

Table 7: Python data services stack

AI, payments and operations	Role
z-ai-web-dev-sdk (GLM series)	Scoring explanations, message and proposal generation, sales assistant
Stripe	Global subscriptions, checkout and webhooks
Razorpay	Indian payment rails and invoices
Vitest, Testing Library, MSW	Unit, component and API mock testing
Sentry	Error tracking and release health
Scripted security scans and backups	Repository scanning plus backup and restore scripts

Table 8: AI, payments and operations stack

8. Delivery Discipline

Feature lists win meetings; delivery discipline ships products. These are the working rules the codebase follows today. They are not aspirations written for this document; each row names where the practice lives in the repository.

Practice	How it shows up in the repo
Typed end to end	TypeScript across front end and API. No untyped boundaries between modules.
Validate at the edge	Zod schemas guard external input before it reaches business logic.
Migrations, not edits	Schema changes go through Prisma tooling, with a migration safety annotation system already written into the schema for the PostgreSQL move.
Least privilege by default	Roles and organization scoping enforced in route handlers. Credits and entitlements are checked server side before any paid action.
Audit what matters	Sensitive actions append to an immutable activity and audit trail with actor, action and timestamp.
Test what breaks	Vitest with Testing Library and MSW covers the flows that have broken before, including auth and billing paths.
Watch it live	Health endpoints, structured logs and Sentry feed one operational view.
Recover on purpose	Backup and restore scripts exist and are exercised. Secrets persist outside the ephemeral environment with restrictive file permissions.

Table 9: Engineering practices and where they live

The discipline is deliberately boring, and boring is the point. It is what allows a small team to carry a system of this size: the codebase currently stands at more than one thousand source files and three hundred twenty six thousand lines, and a new engineer can still find any feature, trace its data and read its tests within a single day of arrival.

9. Field Notes · Verified

These notes are recorded from build experience, not assembled from a deck. Each one cost the team real time before it hardened into a rule. We share them because they say more about how this product is built than any feature table can.

Note 1. OAuth redirect URIs are exact strings. Google rejects a redirect_uri that differs by even one path segment. We register one canonical callback path and every initiation route resolves its redirect_uri to that exact value. Internal hostnames are rejected before any external URL is constructed, so a gateway can never leak a private address into an OAuth flow.

Note 2. Session secrets must be random and must survive restarts. A predictable signing key breaks the trust model, and a rotated key logs every active user out at once. Signing secrets are now generated

randomly and persisted outside the application directory with restrictive file modes, so sessions survive redeployments without weakening security.

Note 3. Email configuration fails silently. A mailer that reports configured while every send fails is worse than one that reports unconfigured, because nobody looks for the problem. Credential resolution now skips placeholder values across every supported alias, so a test password can never masquerade as a real one.

Note 4. Gateways rewrite Host headers. Behind a TLS proxy the application sees an internal host rather than the public domain. The platform therefore derives public URLs from browser supplied origin headers, validates them against a public host rule and falls back to the canonical production domain when validation fails.

Note 5. Plan the database move inside the schema. Every field in our schema carries a migration annotation and every table sits in a dependency ordered phase list. The move from SQLite to PostgreSQL is therefore a planned operation with known backfill points, not an archaeology project discovered midway.

Note 6. Scores need explanations. A number without reasoning does not get trusted, and an untrusted score gets ignored. Each composite score ships with AI written reasoning that cites the signals behind it, which is what turned scoring from a dashboard feature into a daily working habit for the team.

Note 7. Add the boring walls early. Rate limiting, audit logging and GDPR data endpoints were built before launch features rather than after the first incident. Retrofitting walls into a moving system costs roughly three times the effort of pouring them with the foundation.